Table of Contents

An accelerated alternating direction method of multiplier for MRI with TV regularisation....1
Problem Statement....1
Augmented Lagrangian....2
s-update (quadratic problem)....2
t - update....5
Proximal Operator....5
Coding....7
MRI DATA - masking , Fourier Transform & Inverse Fourier Tranform....7
Gradient and Divergence....9
Gradient....9
Divergence....10
Precompute the Fourier Laplacian symbol....11
Mathematically....11
S-update Function....12
Test The S-Update Function....13
T-update....14
Test The T-Update Function....15
Full - ADMM Loop....16

An accelerated alternating direction method of multiplier for MRI with TV regularisation

Team D-1 :

Arjun - CB.SC.U4AIE24305

Dhakshin - CB.SC.U4AIE24313

Kneev S Jain - CB.SC.U4AIE24364

Sanjay Kumar S - CB.SC.U4AIE24354

Total Variation(TV) regularisation is a common technique to improve the sparsity of image,and the Alternating Direction Multiplier Method(ADMM)plays a key role in the variational image processing problem .

Problem Statement

$$y = MS + e$$

$M = RF$, M $\Longrightarrow$ a linear bound operator, such as a blur operator or a sampling matrix.

$$RFs \approx y$$

$$\min \frac{1}{2}||RFs - y||_2^2 + \mu||s||_{tv} + \frac{\epsilon}{2}||\nabla s||_2^2$$

where $||s||_{TV} = ||\nabla s||_1$

if s is an vector , then $(\nabla s)_i = (s_{i+1} - s_i)$, if s is an image, then

$(\nabla s)_{i,j} = (s_{i+1,j} - s_{i,j}, s_{i,j+1} - s_{i,j})$

In the paper, $\min \frac{1}{2}||RFs - y||_2^2 + \mu||s||_{tv} + \frac{\epsilon}{2}||\nabla s||_2^2$ has been considered to solve the TV regularised MRI problem,but it is difficult to solve it directly ,so we transform it into the form of a constrained optimization problem .

$$\min_{s,t} \frac{1}{2}||RFs - y||_2^2 + \mu||t||_1 + \frac{\epsilon}{2}||t||_2^2$$

$$s.t \quad t = \nabla s$$

- s is the image (vectorize or 2-D array),
- F is the 2-D Fourier transform operator
- R is the sampling (mask) operator in k-space (binary; $R^T R$ is a diagonal operator in k-space)
- t is the auxillary variable for the gradient
- ∇ is the (finite-difference) gradient operator producing a 2-channel field(horizontal,vertical)
- $\mu > 0$ is TV regularization weight.
- $\epsilon > 0$ a small quadratic regularization.
- y is the k-space

We'll use **ADMM** with penalty parameter $\beta > 0$. Introduce Lagrange multiplier λ (same shape as t)

Augmented Lagrangian

$$L_\rho(s, t, \lambda) = \frac{1}{2}||RFs - y||_2^2 + \mu||t||_1 + \frac{\epsilon}{2}||t||_2^2 + \langle \lambda, t - \nabla s \rangle + \frac{\beta}{2}||t - \nabla s||_2^2$$

$\langle \cdot, \cdot \rangle$ denotes the real inner product

s-update (quadratic problem)

$$\min_{s,t} \frac{1}{2}||RFs - y||_2^2 - \langle \lambda^k, \nabla s \rangle + \frac{\beta}{2}||t - \nabla s||_2^2$$

lets first expand the last term $||t - \nabla s||_2^2$

$$||t - \nabla s||_2^2 = ||\nabla s||_2^2 - 2\langle t^k, \nabla s \rangle + ||t^k||_2^2$$

$$\frac{\beta}{2}||t - \nabla s||_2^2 = \frac{\beta}{2}(||\nabla s||_2^2 - 2\langle t^k, \nabla s \rangle + ||t^k||_2^2)$$

Combine the linear terms in ∇s :

$$-\langle \lambda^k, \nabla s \rangle + \frac{\beta}{2}(-2\langle t^k, \nabla s \rangle) = -\langle \lambda^k + \beta t^k, \nabla s \rangle$$

So,

$$\min_{s,t} \frac{1}{2}||RFs - y||_2^2 - \langle \lambda^k, \nabla s \rangle + \frac{\beta}{2}||t - \nabla s||_2^2 \Longrightarrow \min_{s,t} \frac{1}{2}||RFs - y||_2^2 + \frac{\beta}{2}||\nabla s||_2^2 - \langle \lambda^k + \beta$$

Partial Derivative $\frac{\mathrm{d}}{\mathrm{d}s} L_\rho$

$$F^T R^T (RFs) - F^T R^T y + \beta \nabla^T \nabla s - \nabla^T (\lambda^k + \beta t^k) = 0$$

$$F^T R^T (RFs) + \beta \nabla^T \nabla s - \nabla^T (\lambda^k + \beta t^k) = F^T R^T y$$

$$(F^T R^T RF + \beta \nabla^T \nabla) s = F^T R^T y + \nabla^T (\lambda^k + \beta t^k)$$

$$Q := F^T R^T RF + \beta \nabla^T \nabla ,$$

$q := F^T R^T y + \nabla^T(\lambda^k + \beta t^k)$.

so $Qs = q$

$F^T R^T RF$ is a diagonal in the frequency domain because R is a pointwise mask in k-space.

$\nabla^T \nabla$ is a convulutional operator . therefore diagonlized by the Fourier transform .

Thus Q is diagonal in the Fourier domain .

wkt $F^T = F^{-1}$, $F * F^{-1} = I$

Apply Fourier tranform on both sides on $(F^T R^T RF + \beta \nabla^T \nabla)s = F^T R^T y + \nabla^T(\lambda^k + \beta t^k)$

we get , $(R^T RF + \beta F \nabla^T \nabla)s = R^T y + F \nabla^T(\lambda^k + \beta t^k)$

$(R^T R + \beta F \nabla^T \nabla F^T)Fs = R^T y + F \nabla^T(\lambda^k + \beta t^k)$

$(R^T R + \beta F \nabla^T \nabla F^T)\hat{s} = R^T y + F \nabla^T(\lambda^k + \beta t^k)$, where $\hat{s} = Fs$

More Compactly : $(mask + \beta \Gamma)\hat{s} = \hat{q}$

$R^T R \equiv mask$ (diagonal in k-space; 1 at sampled frequencies, 0 elsewhere).

$\Gamma \equiv F \nabla^T \nabla F^T$ (diagonal **Laplacian Symbol**(convolutional operator))

$\hat{q} \equiv R^T y + F \nabla^T(\lambda^k + \beta t^k)$

and that's why the solution is just elementwise division in k-space:

$$\hat{s} = \frac{\hat{q}}{mask + \beta \Gamma}, \qquad \hat{s} = Fs$$

t - update

The t - dependant part of L_ρ is

$$\mu ||t||_1 + \frac{\epsilon}{2}||t||_2^2 + \langle \lambda^k, t - \nabla s^{k+1} \rangle + \frac{\beta}{2}||t - \nabla s^{k+1}||_2^2$$

$$\langle \lambda^k, t - \nabla s^{k+1} \rangle \implies \langle \lambda^k, \nabla s^{k+1} \rangle - \langle \lambda^k, t \rangle$$

We only need terms dependant on t and rest are just constants

$$\langle \lambda^k, t \rangle + \frac{\beta}{2}||t - \nabla s^{k+1}||_2^2 = \frac{\beta}{2}||t - (\nabla s^{k+1} - \frac{1}{\beta}\lambda^k)||_2^2 + const$$

$$t^{k+1} = arg \min_t \mu ||t||_1 + \frac{\epsilon}{2}||t||_2^2 + \frac{\beta}{2}||t - (\nabla s^{k+1} - \frac{1}{\beta}\lambda^k||_2^2$$

Here we need to know about **Proximal Operator**

Proximal Operator

A Proximal Operator is a way to minimize functions of the form

$$pro\ x_{\alpha f}(z) = arg \min_x f(x) + \frac{1}{2\alpha}||x - z||_2^2$$

It means: take a vector z, move it closer to minimizing f, but don't go too far - balance closeness to z and regularization f.

- if $f(x) = \mu ||x||_1$, the proximal operator is **shrinkage(soft-thresholding).**
- if $f(x) = \frac{\epsilon}{2}||x||_2^2$, the proximal operator is just a quadratic shrink.

here, our t-subproblem is exactly a proximal operator of the function

$$f(t) = \mu ||t||_1 + \frac{\epsilon}{2}||t||^2$$

applies to $z = \nabla s^{k+1} - \frac{1}{\beta}\lambda^k$, with scaling $\frac{1}{\beta}$

$$\min_t \quad \mu |t| + \frac{\epsilon}{2}t^2 + \frac{\beta}{2}(t - z)^2$$

Combine Quadratics

$$\Longrightarrow \quad \mu |t| + \frac{\beta + \epsilon}{2}(t - \frac{\beta}{\beta + \epsilon}z)^2 + const$$

This is exactly a shrinkage problem: minimize $|t|$ plus a squared distance. The solution is:

$$t^* = shrink \quad (\frac{\beta}{\beta + \epsilon}z, \frac{\mu}{\beta + \epsilon})$$

where shrink is the **soft-threshold** fucntion :

$$shrink \quad (x, \tau) = sign \quad (x) max \quad (|x| - \tau, 0)$$

Since the cost is seperable over components (each pixel and gradient direction), the vector case just applies elementwise

$$t^{k+1} = shrink \quad (\frac{\beta}{\beta + \epsilon}(\nabla s^{k+1} - \frac{1}{\beta}\lambda^k), \frac{\mu}{\beta + \epsilon})$$

WHY SHRINKAGE ?

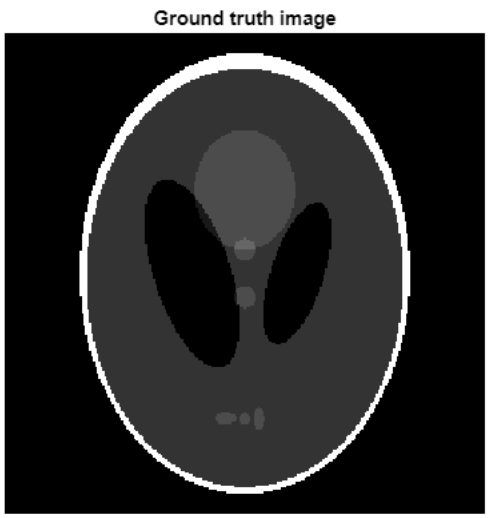
- The $\mu ||t||1$ term encourages sparsity (many gradients become exactly 0).
- The quadratic terms (ϵ, β) prevent instability and keep things convex.
- The combination results in a **soft-thresholding operator:** values below a threshold vanish, large values shrink a bit.

This is exactly what we want: MRI images are piecewise smooth $\rightarrow$ many gradients are near zero $\rightarrow$ soft - thresholding enforces this.

Coding

MRI DATA - masking , Fourier Transform & Inverse Fourier Tranform

```
%% Step 0: Forward model & Zero-filled reconstruction

% Image size
nx = 256; ny = 256;

% Ground truth image (Shepp-Logan phantom)
s_true = phantom('Modified Shepp-Logan', nx);

% Show the ground truth
figure; imshow(s_true,[]); title('Ground truth image');
```




```
% Define centered FFT and iFFT
fft2c  = @(x) fftshift(fft2(ifftshift(x))) / sqrt(nx*ny);
ifft2c = @(x) fftshift(ifft2(ifftshift(x))) * sqrt(nx*ny);

% Full k-space data
y_full = fft2c(s_true);

% Undersampling mask (Cartesian lines)
% n_lines = 70;                % number of sampled k-space lines
% mask = false(nx,ny);
% cols = randperm(ny, n_lines);
```

```
% mask(:, cols) = true;
%-----random sampling...
mask1 = createMask("dummy",256);
% Apply mask to k-space
y = mask1 .* y_full;
```

```
size(y_full)
```

```
ans = 1×2
   256   256
```

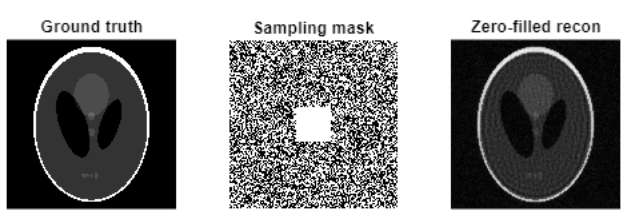
```
size(mask1)
```

```
ans = 1×2
   256   256
```

```
unique(mask1)
```

```
ans = 2×1
     0
     1
```

```
% Zero-filled reconstruction
s_zf = ifft2c(y);

% Display results
figure;
subplot(1,3,1); imshow(s_true,[]); title('Ground truth');
subplot(1,3,2); imshow(mask1,[]); title('Sampling mask');
subplot(1,3,3); imshow(abs(s_zf),[]); title('Zero-filled recon');
```




```
% Compute PSNR (assuming image scaled in [0,1])
mse = mean((abs(s_zf(:))-s_true(:)).^2);
```

```
psnr_zf = 10*log10(1/mse);
fprintf('PSNR of zero-filled recon: %.2f dB\n', psnr_zf);
```

```
PSNR of zero-filled recon: 24.35 dB
```

$Ifftshift(x)$ - Moves the *zero frequency* (DC) from the center **to the top-left corner** before doing the FFT.

$fft2()$ - Performs **Discrete Fourier Transform** on x(shifted) .

$ifft2()$ - Performs **Inverse Discrete Fourier Transform.**

$fftshift()$ - Moves the *zero frequency* to the **center** (MRI-style k-space layout).

$numel(x)$ - total number of pixels

$/sqrt(numel(x))$ - Normalizes the FFT so that `ifft2c(fft2c(x))` returns exactly x without scaling changes.

$fft2c \Longrightarrow image \longrightarrow K - space$

$ifft2c \longrightarrow K - space \rightarrow Image$

Gradient and Divergence

These are the Total variation operators - super important because $t = \nabla s$.

Gradient ∇

In 2D images, the gradient computes finite difference along x (horizontal) and y (vertical):

$\nabla s = (g_x, g_y)$

$g_x(i, j) = s(i, j+1) - s(i, j) \quad , \quad g_y(i, j) = s(i+1, j) - s(i, j)$

- g_x = how much the image changes left-right.
- g_y = how much the image changes up-down.

so $t = (gx, gy)$ is a **2-channel image** .

Divergence ∇^T

Divergence is the adjoint operator of the gradient.

It maps a 2-channel vector field back to a scalar image.

$$\nabla^T(p_x, p_y)(i, j) = (p_x(i, j) - p_x(i, j-1)) + (p_y(i, j) - p_y(i-1, j))$$

This is like the opposite of ∇ : it sums "flows" into each pixel.

It is also the **Negative discrete Laplacian** when as $\nabla^T \nabla s$

```
%% Step 1: Gradient and Divergence operators

% Gradient operator
function [gx, gy] = grad_tv(x)
    gx = circshift(x,[0 -1]) - x;   % horizontal difference
    gy = circshift(x,[-1 0]) - x;   % vertical difference
end

% Divergence operator (
function d = div_tv(px, py)
    d = (px - circshift(px,[0 1])) + (py - circshift(py,[1 0]));
end
```

$circshift \quad (x, [0 \; -1]) \rightarrow$ shifts the image left by 1 column.

then $-x$ gives the forward difference horizontally.

Then WE DO THE ADJOINTNESS CHECK !

$$\langle \nabla s, p \rangle = \langle s, -\nabla^T p \rangle$$

```
% Test adjointness
x = randn(64);                    % random image
px = randn(64); py = randn(64);   % random vector field

[gx,gy] = grad_tv(x);

lhs = sum(sum(gx.*px + gy.*py));        % <∇x, p>
rhs = -sum(sum(x .* div_tv(px,py)));    % <x, -∇ᵀp>

fprintf('Adjointness check: difference = %.2e\n', abs(lhs-rhs));
```

```
Adjointness check: difference = 1.42e-14
```

Precompute the Fourier Laplacian symbol Γ

From the s-update Derivation we had in internal link

$(R^T R + \beta F \nabla^T \nabla F^T)\hat{s} = R^T y + F \nabla^T (\lambda^k + \beta t^k)$, where $\hat{s} = Fs$

$$\implies (mask + \beta \Gamma)\hat{s} = \hat{q}$$

where $\Gamma = F \nabla^T \nabla F^T$.

Γ is the Fourier symbol of the Laplacian Operator .

$\hat{s} = \dfrac{\hat{q}}{mask + \beta \Gamma}$, so Γ tells us how the **Gradient operator looks in Fourier Space**.

Γ Mathematically

wkt

- $g_x(i, j) = s(i, j+1) - s(i, j)$, $g_y(i, j) = s(i+1, j) - s(i, j)$
- Its Adjoint ∇^T leads to the Laplacian: $\nabla^T \nabla s \approx -(\Delta_x + \Delta_y)s$.

In Fourier domain :

$$|G_x(w_x)|^2 = e^{iw_x} - 1|^2 = 2 - 2\cos(w_x)$$

$$|G_y(w_y)|^2 = e^{iw_y} - 1|^2 = 2 - 2\cos(w_y)$$

So: $\Gamma(w_x, w_y) = |G_x|^2 + |G_y|^2 = (2 - 2\cos(w_x)) + (2 - 2\cos(w_y))$

```
%% Step 2: Precompute Fourier Laplacian symbol Γ
```

```
function Gamma = laplacian_symbol_centered(nx, ny)
    % frequency grids in [-pi, pi) to match fftshifted k-space
    kx = 2*pi * ((-floor(ny/2)):(ceil(ny/2)-1)) / ny;  % horiz freq (columns)
    ky = 2*pi * ((-floor(nx/2)):(ceil(nx/2)-1)) / nx;  % vert   freq (rows)
    [WX, WY] = meshgrid(kx, ky);

    Gx = exp(-1i*WX) - 1;
    Gy = exp(-1i*WY) - 1;

    Gamma = abs(Gx).^2 + abs(Gy).^2;  % ≥ 0, centered like fft2c
end
```

we see an image that's dark in the center (low frequency, small Laplacian penalty) and bright at edges (high frequency, strong penalty). This matches the intuition: **Laplacian penalizes high-frequency variations**.

S-update Function

$$(mask + \beta\Gamma)\hat{s} = \hat{q} \quad , \quad \hat{q} = R^T y + F\nabla^T(\lambda + \beta t) \quad , \quad \hat{s} = Fs$$

```
function s = admm_update_s(y, mask, beta, tx, ty, lamx, lamy, Gamma, fft2c, ifft2c)
% s-update solving (mask + beta*Gamma) .* S_hat = Q_hat in centered k-space

    % (1) build ∇^T(λ + β t) in spatial domain
    px = lamx + beta * tx;
    py = lamy + beta * ty;
    rhs_spatial = div_tv(px, py);

    % (2) go to k-space and add R^T y (mask .* y, both centered)
    rhs_k = fft2c(rhs_spatial);
    q_hat = mask .* y + rhs_k;

    % (3) denominator (must be strictly > 0 at sampled DC)
    denom = mask + beta * Gamma;

    % --- DEBUG: quick sanity prints
    dmin = min(denom(:)); dmax = max(denom(:));
    fprintf('[s-update] denom: min=%g  max=%g  (mask sum=%d)\n', dmin, dmax,
nnz(mask));
    if dmin <= 0
        warning('Denominator has non-positive entries! Check mask/Gamma
alignment.');
    end

    % guard any leftover zeros (shouldn't happen if DC is sampled)
    denom(denom == 0) = 1e-12;

    % (4) elementwise solve in k-space, then back to image
```

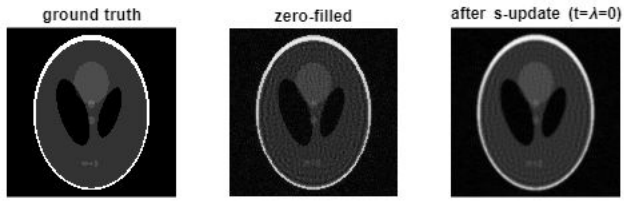
```
    s_hat = q_hat ./ denom;
    s = ifft2c(s_hat);
end
```

Test The S-Update Function

```
% --- sizes & data
nx = 256; ny = 256;
s_true = phantom('Modified Shepp-Logan', nx);

fft2c  = @(x) fftshift(fft2(ifftshift(x))) / sqrt(nx*ny);
ifft2c = @(x) fftshift(ifft2(ifftshift(x))) * sqrt(nx*ny);

y_full = fft2c(s_true);

% --- make a Cartesian mask (ensure DC is kept)
% --- undersampling mask (Cartesian columns), then CENTER it
% n_lines = 70;
% mask = false(nx,ny);
% cols = randperm(ny, n_lines);
% mask(:, cols) = true;
%-----random sampling...
% keep DC for stability
y = mask1 .* y_full;

% --- precompute Gamma
Gamma = laplacian_symbol_centered(nx, ny);  % from Step 2

% --- initialize ADMM vars (t, lambda) to zero
tx = zeros(nx,ny); ty = zeros(nx,ny);
lamx = zeros(nx,ny); lamy = zeros(nx,ny);

% --- choose beta
beta = 1.0;

% --- compute s-update once
s_once = admm_update_s(y, mask1, beta, tx, ty, lamx, lamy, Gamma, fft2c, ifft2c);
```

```
[s-update] denom: min=0.393585  max=9  (mask sum=34306)
```

```
% --- compare to zero-filled
s_zf = ifft2c(y);

% --- show
figure;
subplot(1,3,1); imshow(s_true,[]); title('ground truth');
subplot(1,3,2); imshow(abs(s_zf),[]); title('zero-filled');
subplot(1,3,3); imshow(abs(s_once),[]); title('after s-update (t=\lambda=0)');
```




```
% --- PSNRs
mse_zf   = mean((abs(s_zf(:))   - s_true(:)).^2);
mse_once = mean((abs(s_once(:)) - s_true(:)).^2);
psnr_zf   = 10*log10(1/mse_zf);
psnr_once = 10*log10(1/mse_once);
fprintf('PSNR zero-fill: %.2f dB | after s-update: %.2f dB\n', psnr_zf, psnr_once);
```

```
PSNR zero-fill: 24.35 dB | after s-update: 22.20 dB
```

T-update

$$t^{k+1} = shrink \left(\frac{\beta}{\beta + \epsilon} \left(\nabla s^{k+1} - \frac{1}{\beta} \lambda^{k} \right), \frac{\mu}{\beta + \epsilon} \right)$$

here $t = (t_x, t_y)$, $\nabla s = (g_x, g_y)$, and $\lambda = (\lambda_x, \lambda_y)$

Anisotropic TV (componentwise shrink):

$$shrink(x, \tau) = sign(x) max(|x| - \tau, 0)$$

isotropic TV : shrink the vector magnitude per pixel

```
% -------- Soft-threshold (anisotropic), real or complex-safe
function y = shrink_aniso(x, tau)
    % works for real or complex x
```

```
    mag = abs(x);
    y = max(mag - tau, 0) .* (x ./ max(mag, eps));  % preserve phase
end

% -------- Soft-threshold (isotropic) for 2D vectors (px,py)
% (optional, only if you want isotropic TV later)
function [qx, qy] = shrink_iso(px, py, tau)
    mag = sqrt(abs(px).^2 + abs(py).^2);
    scale = max(mag - tau, 0) ./ max(mag, eps);
    qx = scale .* px;
    qy = scale .* py;
end

function [tx, ty] = admm_update_t(s, lamx, lamy, beta, mu, eps2, grad_tv)

    % ---- 1) gradient of current image
    [gx, gy] = grad_tv(s);                          % ∇ s^{k+1}

    % ---- 2) “center” for prox: grad(s) - (1/beta)*lambda
    zx = gx - lamx / beta;
    zy = gy - lamy / beta;

    % ---- 3) scale inputs and threshold per the closed form
    alpha = beta / (beta + eps2);                   % input scaling
    tau   = mu   / (beta + eps2);                   % threshold

    % ---- 4) anisotropic shrink (componentwise)
    tx = shrink_aniso(alpha * zx, tau);
    ty = shrink_aniso(alpha * zy, tau);
end
function [lamx, lamy] = admm_update_lambda(lamx, lamy, beta, tx, ty, s, grad_tv)
% λ^{k+1} = λ^k + β ( t^{k+1} - ∇ s^{k+1} )
    [gx, gy] = grad_tv(s);
    relax = 1.8;  % between 1.0 and 2.0
    lamx = lamx + relax*beta*(tx - gx);
    lamy = lamy + relax*beta*(ty - gy);

end
```

Test The T-Update Function

```
%% quick sanity for t-update
mu   = 0.01;     % try 1e-2 .. 1e-1
eps2 = 1e-2;     % small ε for stability

% start from s_once you computed previously
[tx, ty] = admm_update_t(s_once, lamx, lamy, beta, mu, eps2, @grad_tv);
```

```
% check that t is sparse-ish (many entries near zero)
fprintf('t nonzeros (>|1e-4|): %d of %d\n', ...
    nnz(abs(tx) > 1e-4) + nnz(abs(ty) > 1e-4), 2*numel(s_once));
```

```
t nonzeros (>|1e-4|): 90009 of 131072
```

```
% one lambda update (not yet iterating fully)
[lamx, lamy] = admm_update_lambda(lamx, lamy, beta, tx, ty, s_once, @grad_tv);
```

```
%-----random sampling...
function M = createMask(str,n)
    % str = "Vertical, Horizontal, radial"
    M = zeros(n);
    R = rand(n);
    M(R <= 0.5) = 1;
    df = 0.1*n;
    dr = round(n/2 - df):round(n/2 + df);
    dc = round(n/2 - df):round(n/2 + df);
    M(dr,dc) = 1;
end
```

Full - ADMM Loop

```
function [s, psnr_list] = mri_admm(y, mask, s_true, params, fft2c, ifft2c)
    [nx, ny] = size(y);

    % --- precompute Laplacian symbol
    Gamma = laplacian_symbol_centered(nx, ny);

    % --- init variables
    s = ifft2c(y);                  % start from zero-filled
    tx = zeros(nx,ny); ty = zeros(nx,ny);
    lamx = zeros(nx,ny); lamy = zeros(nx,ny);

    psnr_list = zeros(params.n_iter,1);

    % --- main ADMM loop
    for k = 1:params.n_iter

        % (1) s-update
        s = admm_update_s(y, mask, params.beta, tx, ty, lamx, lamy, Gamma, fft2c,
ifft2c);

        % (2) t-update
        [tx, ty] = admm_update_t(s, lamx, lamy, params.beta, params.mu,
params.eps2, @grad_tv);
```

```
        % (3) λ-update
        [lamx, lamy] = admm_update_lambda(lamx, lamy, params.beta, tx, ty, s,
@grad_tv);

        % monitor PSNR (if ground truth provided)
        if ~isempty(s_true)
            mse = mean((abs(s(:)) - s_true(:)).^2);
            psnr_list(k) = 10*log10(1/mse);
            fprintf('Iter %3d: PSNR = %.2f dB\n', k, psnr_list(k));
        end
    end
end
```

```
%% demo script for full ADMM MRI reconstruction
tic
nx = 256; ny = 256;
s_true = phantom('Modified Shepp-Logan', nx);

fft2c  = @(x) fftshift(fft2(ifftshift(x))) / sqrt(nx*ny);
ifft2c = @(x) fftshift(ifft2(ifftshift(x))) * sqrt(nx*ny);

% full k-space
y_full = fft2c(s_true);

% undersampling mask
% n_lines = 70;
% mask = false(nx,ny);
% cols = randperm(ny, n_lines);
% mask(:, cols) = true;

% ensure DC is sampled (center pixel in fftshifted coords)
mask1(round(nx/2), round(ny/2)) = true;
fprintf('mask sum = %d\n', nnz(mask1));
```

```
mask sum = 34306
```

```
y = mask1 .* y_full;

% parameters
% Try a smaller value to reduce oscillations
params.beta = 0.8; % Or try 0.5, 0.2 etc.
params.mu     = 0.01;
params.eps2   = 1e-2;
params.n_iter = 300;

% run ADMM
[s_rec, psnr_list] = mri_admm(y, mask1, s_true, params, fft2c, ifft2c);
```

```
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter   1: PSNR = 22.47 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter   2: PSNR = 20.04 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter   3: PSNR = 22.90 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter   4: PSNR = 20.52 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter   5: PSNR = 21.17 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter   6: PSNR = 21.90 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter   7: PSNR = 20.95 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter   8: PSNR = 21.48 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter   9: PSNR = 21.35 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  10: PSNR = 21.34 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  11: PSNR = 21.33 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  12: PSNR = 21.44 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  13: PSNR = 21.31 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  14: PSNR = 21.39 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  15: PSNR = 21.37 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  16: PSNR = 21.37 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  17: PSNR = 21.36 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  18: PSNR = 21.39 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  19: PSNR = 21.37 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  20: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  21: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  22: PSNR = 21.37 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  23: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  24: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  25: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  26: PSNR = 21.37 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  27: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  28: PSNR = 21.37 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  29: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  30: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  31: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  32: PSNR = 21.37 dB
```

```
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  33: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  34: PSNR = 21.37 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  35: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  36: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  37: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  38: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  39: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  40: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  41: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  42: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  43: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  44: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  45: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  46: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  47: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  48: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  49: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  50: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  51: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  52: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  53: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  54: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  55: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  56: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  57: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  58: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  59: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  60: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  61: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  62: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  63: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  64: PSNR = 21.38 dB
```

```
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  65: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  66: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  67: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  68: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  69: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  70: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  71: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  72: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  73: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  74: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  75: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  76: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  77: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  78: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  79: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  80: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  81: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  82: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  83: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  84: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  85: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  86: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  87: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  88: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  89: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  90: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  91: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  92: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  93: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  94: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  95: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  96: PSNR = 21.38 dB
```

```
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  97: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  98: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter  99: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 100: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 101: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 102: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 103: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 104: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 105: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 106: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 107: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 108: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 109: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 110: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 111: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 112: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 113: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 114: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 115: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 116: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 117: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 118: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 119: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 120: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 121: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 122: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 123: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 124: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 125: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 126: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 127: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 128: PSNR = 21.38 dB
```

```
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 129: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 130: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 131: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 132: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 133: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 134: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 135: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 136: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 137: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 138: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 139: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 140: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 141: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 142: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 143: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 144: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 145: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 146: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 147: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 148: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 149: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 150: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 151: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 152: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 153: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 154: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 155: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 156: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 157: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 158: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 159: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 160: PSNR = 21.38 dB
```

```
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 161: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 162: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 163: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 164: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 165: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 166: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 167: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 168: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 169: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 170: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 171: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 172: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 173: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 174: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 175: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 176: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 177: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 178: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 179: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 180: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 181: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 182: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 183: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 184: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 185: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 186: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 187: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 188: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 189: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 190: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 191: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 192: PSNR = 21.38 dB
```

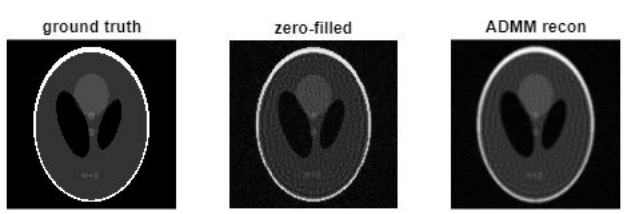
```
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 193: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 194: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 195: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 196: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 197: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 198: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 199: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 200: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 201: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 202: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 203: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 204: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 205: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 206: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 207: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 208: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 209: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 210: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 211: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 212: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 213: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 214: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 215: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 216: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 217: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 218: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 219: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 220: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 221: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 222: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 223: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 224: PSNR = 21.38 dB
```

```
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 225: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 226: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 227: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 228: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 229: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 230: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 231: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 232: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 233: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 234: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 235: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 236: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 237: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 238: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 239: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 240: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 241: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 242: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 243: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 244: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 245: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 246: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 247: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 248: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 249: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 250: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 251: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 252: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 253: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 254: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 255: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 256: PSNR = 21.38 dB
```

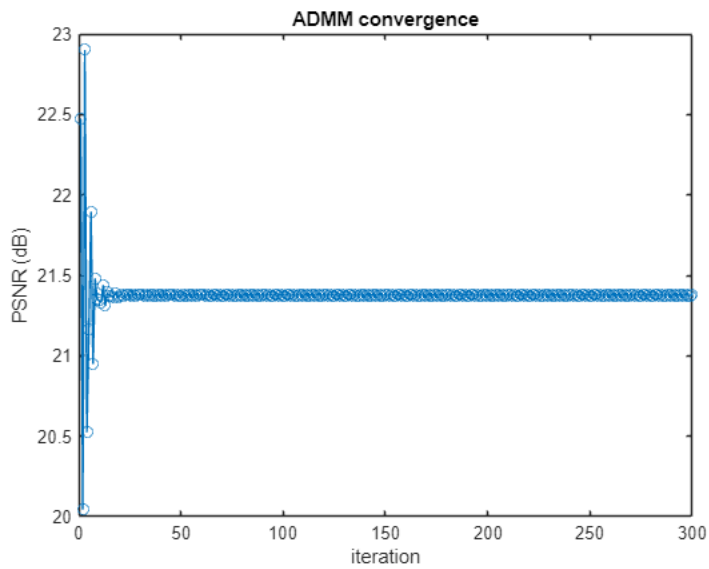
```
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 257: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 258: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 259: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 260: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 261: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 262: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 263: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 264: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 265: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 266: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 267: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 268: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 269: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 270: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 271: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 272: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 273: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 274: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 275: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 276: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 277: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 278: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 279: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 280: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 281: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 282: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 283: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 284: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 285: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 286: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 287: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 288: PSNR = 21.38 dB
```

```
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 289: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 290: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 291: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 292: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 293: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 294: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 295: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 296: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 297: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 298: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 299: PSNR = 21.38 dB
[s-update] denom: min=0.314868  max=7.4  (mask sum=34306)
Iter 300: PSNR = 21.38 dB
```

```
% display
figure;
subplot(1,3,1); imshow(s_true,[]); title('ground truth');
subplot(1,3,2); imshow(abs(ifft2c(y)),[]); title('zero-filled');
subplot(1,3,3); imshow(abs(s_rec),[]); title('ADMM recon');
```




```
% convergence plot
figure;
plot(psnr_list,'-o');
xlabel('iteration'); ylabel('PSNR (dB)');
title('ADMM convergence');
```




```
toc
```

```
Elapsed time is 4.156034 seconds.
```